PoCoTo - An Open Source System for Efficient Interactive Postcorrection of OCRed Historical Texts

Thorsten Vobl*
CIS
University of Munich
thorsten.vobl@gmail.com

Annette Gotscharek†
CIS
University of Munich
annettegotscharek@t-online.de

Uli Reffle‡
CIS
University of Munich
uli@reffle.de

Christoph Ringlstetter
Gini
c.ringlstetter@gini.net

Klaus U. Schulz
CIS
University of Munich
schulz@cis.uni-muenchen.de

ABSTRACT

When applied to historical texts, OCR engines often produce a non-negligible number of OCR errors. For research in the Humanities, text mining and retrieval, the option is important to improve the quality of OCRed historical texts using interactive postcorrection. We describe a system for interactive postcorrection of OCRed historical documents developed in the EU project IMPACT. Various advanced features of the system help to efficiently correct texts. Language technology used in the background takes orthographic variation in historical language into account. Using this knowledge, the tool visualizes possible OCR errors and series of similar possible OCR errors in a given input document. Error series can be corrected in one shot. Practical user tests in three major European libraries have shown that the system considerably reduces the time needed by human correctors to eliminate a certain number of OCR errors. The system has been published as an open source tool under GitHub.

Categories and Subject Descriptors

I.7.5 [**Document Capture**]: Optical character recognition (OCR)

General Terms

Design, Performance

Keywords

error correction, decision support, user interfaces

*Author has left CIS.
†Author has left CIS.
‡Author has left CIS.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
DATeCH 2014 May 19 - 20 2014, Madrid, Spain
Copyright 2014 ACM 978-1-4503-2588-2/14/05 $15.00
http://dx.doi.org/10.1145/2595188.2595197.

1. INTRODUCTION

In the wake of Google books and other digitization programs, an immense number of historical books is/has been scanned, captured via Optical Character Recognition (OCR) and brought to the internet in textual form. In many cases the OCR quality is low, and because of that a major part of OCRed documents are of limited use for researchers in the Humanities: text mining and retrieval are fragmentary [12, 13]. Since fully automated methods for postcorrection are not reliable, there is an obvious interest in the Humanities in efficient interactive postcorrection. Selected and important texts/corpora can be lifted to a much higher level of accuracy. Here the user experience offered by a system for interactive postcorrection has a major influence on time and efforts needed for improving accuracy.

In this paper we describe a system for interactive postcorrection of OCRed historical texts developed in the EU project IMPACT. Advanced features of the system enable efficient detection and correction of errors. Special language technology *aware of historical language variation* is used to point users to "suspicious" tokens of the OCR input text, and to offer plausible corrections, also *including words in historical orthography.* A graphical user interface (GUI) provides parallel views on OCRed text and underlying images. Image parts corresponding to suspicious tokens are highlighted. In this way it is easy to check if a suspicious token in fact represents an error, and to correct errors. As a unique feature, the system also computes possible *series of similar OCR errors and possible simultaneous corrections for all tokens of such a series.* A *special concordance view* in parallel presents (i) the textual context of each member of a given series in the OCR output and (ii) the image parts corresponding to each context, and joint correction suggestions for all entries. Users can see immediately if a one-shot batch correction of all members of an error series is possible.

The benefit offered by the system has been tested not only in our research lab (CIS/LMU Munich), but also at three major European libraries. Languages covered in these tests are historical German, Spain, and Dutch. The time needed by human users for correcting a certain number of errors in a given OCRed corpus was measured using the tool in full/baseline mode. Results show that PoCoTo postcorrection system even in base mode (without batch correction) helps to efficiently correct texts. Using batch correction of error series, a larger number of errors can be eliminated in

a shorter amount of time.

The paper has the following parts. We start with a brief survey on related work in Section 2. Section 3 offers a non-technical introduction to the system, explaining its use and the main features. Section 4 presents results and numbers from our evaluation experiments. Section 5 describes the language technology and resources used in the background and explains basic features of the implementation. In the Conclusion 6 we sum up, point to our vision of a PoCoTo community and add remarks on future work.

2. RELATED WORK

There is a huge literature on automated and interactive text correction, see [6] for an excellent older survey.

Cleansing of historical OCRed texts. Recent work by Reynaert addresses non-interactive postcorrection of OCR results including historical texts [10]. Similarities between words and word frequencies observed in an input text are used for filtering. Evaluation principles are discussed in [9]. Boschetti et al. [1] show how to align the output of several OCR engines on the same image for improving OCR accuracy of classical critical editions.

Reffle and Ringlstetter [8] introduced a statistical method for deriving from an OCRed historical text a profile in terms of conjectured errors, error series, and historical language variants. Special finite-state technology for efficiently generating correction suggestions for garbled tokens of historical texts [7] is built in. Wudtke et al. [15] show how to detect garbage in OCR output on historical documents. Holley [4] compares methods of improving accuracy in historic newspaper digitization programs. and shows in [5] how crowdsourcing can be used to improve OCRed collections. A system based on user collaboration is described by Conteh and Tzadok in [2].

Interactive spelling and OCR correction. Taghva and Stofsky [14] describe an interactive spelling correction system for OCR errors. For suggesting error corrections, the system learns from confusion information at the document level. Strohmaier et al. [11] use a visual interactive system for optimizing lexical postcorrection of OCR results. Garfinkel et al. [3] discuss how to optimize the selection of correction suggestions in interactive spell checking.

3. INTRODUCTION TO POCOTO

A session with the postcorrection system starts with a series of obvious interaction steps where the user loads a set of OCRed texts with associated images and creates a "project". Input resources are held in dedicated subdirectories per project. OCRed input texts come in a special XML format, the result of previous correction sessions is preserved. Our XML input format builds up on the output formats of the OCR engine, currently ABBYYs XML output format or Tesseract's HOCR format.

Base mode - correction in reading order. After the user has loaded a set of OCRed texts together with the corresponding page image files, the base view of the GUI shows in one window (Window 1) text and image *snippet pairs aligned at a word-by-word level.* The user can directly compare each word of the recognition result with the corresponding image region, proceeding in standard reading order. A second window presents a parallel view of the corresponding full page image. This window is used in situations where OCRed words are heavily garbled, the segmentation of the OCR has failed, or sentence/paragraph context is necessary to properly judge correctness of the OCR result or its correction suggestion. When the user activates a word of the OCRed text in Window 1, the corresponding region of the full page image is marked with a red box. This standard view is shown in Figure 3.

If a word snippet is selected, a context menu offers possible correction candidates. The user may choose one of these candidates as the correction or manually edit the OCR output token. An example is shown in Figure 3. We see the editing window which still contains the current OCR result "kormiren" (an OCR error) and the context menu including correction suggestions.

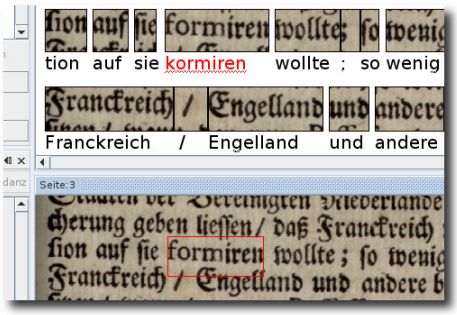


Figure 1: Base mode: tokens of the OCR output including image snippets, as well as a full view of the page can be viewed in parallel.

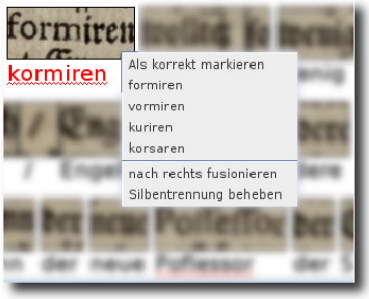


Figure 2: Choosing correction candidates using the context menu of a token.

Special language technology aware of historical language variation and exploiting adaptive treatment of each single input document (see below) is used to automatically detect "suspicious" tokens that represent possible OCR errors. Historical orthographic variants are only treated as suspicious in case of statistical evidence that OCR has garbled the word. Suspicious tokens are marked in Window 1. In this way, the user is quickly led to possible OCR errors.

Advanced mode - batch correction of error series. As a special feature, the system offers a number of advanced mechanisms for efficient *batch correction* of complete "series" of OCR errors. A common feature of these mechanisms is that the system shows all suspicious tokens of such a series in a *list of concordances.* For a given error candidate

- a suspicious token of the text - the concordance shows a text window of the OCR file, containing the token and its neighborhood, and the corresponding region of the parallel image file. In general, a list of concordances presents many suspicious tokens, as well as the individual correction suggestions, giving the user the possibility to correct all tokens of the list in one shot.

The simplest case of such a batch correction is the situation where the user wants to see all occurrences of a suspicious string in the text (cf. Figure 3). In this situation, the series of suspicious tokens consists of all occurrences of the string. All tokens in the concordance list are identical, as are the correction suggestions. The user can decide to correct all tokens of the list, to exclude some members, to use the given correction suggestion or to introduce a new correct word as a common substitute.

A more advanced example of batch correction is based on OCR error types. As an example, consider the situation where the OCR has frequently confused the letters "u" and "n" in distinct words of the input text. Our background language technology ("profiler", s.b.) often detects such cases and the system then suggests to show all tokens of the text where it assumes that the OCR has confused the two letters as a "series" (cf. Figure 4). As before, the user can decide to correct all members of the list, to exclude some members, to use the correction suggestions presented or to introduce a new correct word as a common substitute. More generally, the profiling software is used to compute a *list of possible types of OCR errors* found in the document, estimating the number of occurrences of the error in the text. The user may select a type of OCR error and interact in the way indicated above.

Yet another instance of batch correction is to show in the list of concordances all tokens where the system has "strong evidence" that the token represents an error and the system is "convinced" to know the correct word. Again, evidence is based on previous profiling of the input text.

After each correction interaction the user may store the new version of the text. The system is aware of which token has been corrected, keeping track of the original word. Corrected words of the text are marked green. In this way it is avoided to repeatedly check the same token in distinct sessions of the same project.

4. EVALUATION EXPERIMENTS

In three test series at the Bavarian State Library (BSB), the University of Alicante (UA) and the Dutch National Library (KB) we evaluated the time needed for interactive correction of OCR errors using the PoCoTo system. OCRed documents tested respectively were selected by BSB, UA, and KB, and written in historical German, Spanish, and Dutch. To see which benefits can be obtained when using the advanced features of the postcorrection tool we used two settings at BSB and UA. In a first baseline setting, users only had access to the graphical interface (non-lexical words in the OCR output marked) without any options for batch functionalities. In a second setting with similar other documents, all features were available. Users were free to use batch mode or base mode. The correction time for the full experiments was 90 minutes. Protocols captured the number of corrections after each period of ten minutes.

The results of the experiments at BSB and UA (each using two documents) are displayed in Figures 5 and 6 for BSB and in Figures 7 and 8 for UA. Each line represents one user.

For each measuring point (every ten minutes) the amount of errors corrected is shown. Empty measuring points mean the user forgot to click the save button (User4 had to leave the test early for the first experiment shown in Figure 5). 'Users Full' are the Users that had all the features available, while 'Users Base' are the ones that did not have access to the batch correction features described in Section 3. In this way the base mode can be compared to a standard correction system with the improvement that lines of images and OCR output are aligned visually. This makes it much easier to switch focus between the two.

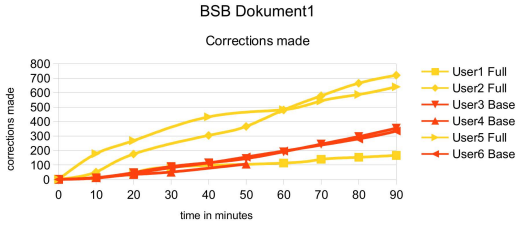


Figure 5: Base mode: tokens of the OCR output including image snippets, as well as a full view of the page can be viewed in parallel. Full mode: all features of PoCoTo available.

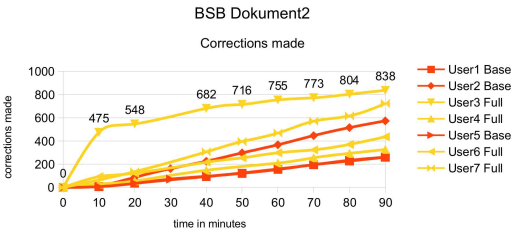


Figure 6: Base mode: tokens of the OCR output including image snippets, as well as a full view of the page can be viewed in parallel. Full mode: all features of PoCoTo available.

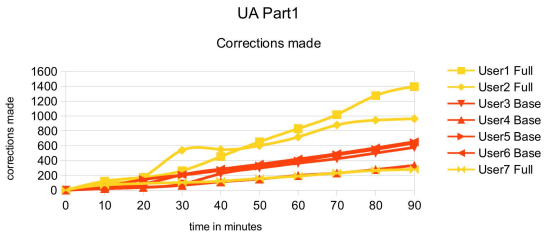


Figure 7: Base mode: tokens of the OCR output including image snippets, as well as a full view of the page can be viewed in parallel. Full mode: all features of PoCoTo available.



Figure 3: Batch processing of all occurrences of erroneous string "Zrankreich".



Figure 4: Batch processing based on identical error patterns (n↦ u).

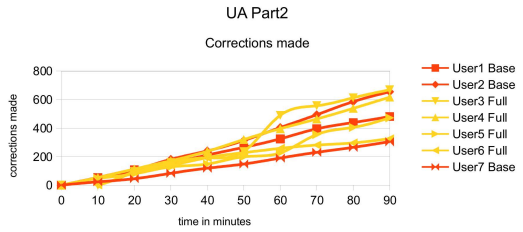


Figure 8: Base mode: tokens of the OCR output including image snippets, as well as a full view of the page can be viewed in parallel. Full mode: all features of PoCoTo available.

For the BSB experiment (Figures 5 and 6) the data show a clear advantage of the full mode. For experiment one, User1 is an outlier: instead of productive use of the functionality he explored the tool. By average the two serious users of the full mode corrected 7.5 errors per minute. For the base mode we had 3.8 per minute. For the second experiment in total the full-feature users average 6.4 corrections per minute, while the base users average 4.0 corrections. The curve of User3 is indicating the real potential of the full-feature mode in the beginning of the second test phase. Taking advantage of some very productive error patterns, the user was able to gather about 500 corrections in only ten minutes

For the UA experiment (Figures 7 and 8) the data of the first experiment also shows a clear picture where in total the full users averaged 9.8 corrections per minute and the base users 6.1. For the second experiment the performance gain of the full mode is not significant. An explanation is that for the full mode, the efficiency of the advanced features depends on the quality of the profile. That means, the accuracy of the correction candidates and of the error patterns computed by the profiling module determines the performance of the batch features of the tool. The curves of User3 and User5 indicate they used the batch correction features only sometimes (notice the bumps in the curve) whereas the rest of the full users show a behavior that would more likely to be expected for a base user, indicating that they very rarely used the advanced correction features.

In general, the individual result a user is able to achieve in such an experiment is influenced by many factors, such as the ease of a user to become acquainted with the tool and use it effectively. Note the relatively short training phase of only 30 minutes for the experiments. Additional issues in the chosen setting are user motivation (especially over the complete experiment) and the extent to which advanced batch correction methods are used.

For the KB we only experimented with the full mode. As in the other experiments, we observed a rather big difference between individual users. While the user with the most corrections reached 9.5 per minute, the average of all users was only 7.14. Values in line with experiments for the other languages. Overall, the data obtained during the experiments in the libraries – supported by user surveys and in-house experiments with students – show that after a short training period with the tool, in most cases, the users with access to the full mode performed significantly better than the users working with a version of the tool that lacked the full capacity of the background language technology. The productivity, which is expected to further rise with complete knowledge of the advanced features of PoCoTo, was between two to three times higher for the motivated users of the full mode. This effect is strengthened if we evaluate the earlier phase of the experiments e.g. if we look at the first 20 minutes. Thanks to bulk corrections with the batch features, the number of corrected tokens in this time frame is factor 5 to 10 higher as compared to the base mode. This advantage of the tool can be exploited to implement a process enabling rapid improvement of a collection within a reasonable cost range.

5. LANGUAGE TECHNOLOGY, IMPLEMENTATION ASPECTS, ACCESS

The postcorrection system calls a built-in webservice to derive for each (multi-page) input document a document-specific language *profile* with two parts. (i) The "global" part of the profile contains (a) a list of all OCR error types (e.g., n ↦ ii) found in the document, ordered by the num-

ber of occurrences in the input document, and (b) a list of all "patterns" for orthographic language variation" observed in the document, ordered by document-frequency. Patterns are rewrite rules (e.g. t $\mapsto$ th) explaining the difference between modern and historical spelling (e.g., modern word "turm", historical spelling "thurm"). Since no ground truth is available, all OCR-errors, patterns, and frequencies are only based on statistical evidence. To obtain statistical estimates, the profiling webservice uses a complex kind of expectation-maximization procedure. Details can be found in [8]. (ii) The "local" part of the profile specifies for each single token of the input document a set of possible "interpretations". The notion of an interpretations relies on the idea of a two-channel model for a token t_{ocr} observed in the OCR output. The OCR channel leads from the correct word of the text t_{text} to the recognized t_{ocr}, possibly introducing OCR errors. The "historical-language-variation" channel leads from a word in modern spelling t_{mod} to the historical variant t_{text}, possibly using patterns. An interpretation for a token t_{ocr} specifies t_{text}, t_{mod}, as well as OCR-errors and patterns used in the derivation. Each interpretation comes with a probability. Interpretations are used to calculate "suspicious" tokens and optimal correction suggestions. Specific resources needed for profiling input documents for a specific language L are: a list of full forms in modern spelling for L with frequencies and a list of patterns that captures historical orthographic variation. The present version of the webservice can treat historical German, English, Dutch, French, and Spanish. PoCoTo itself is a stand-alone client software. The library part of PoCoTo is programmed in C++, the graphical user interface is implemented in Java using the Netbeans Platform framework. The system has been published as an open source tool under GitHub.[1]

6. CONCLUSION

We presented PoCoTo, an open source system for interactive postcorrection of OCRed historical texts. Unique features of the system rely on built-in special language technology for historical language and OCR error recognition. Using this technology, PoCoTo points the user to possible errors and error series and offers actions for efficient batch correction. We are currently trying to build a community for maintaining and improving the tool.

Future work will be focused on several points. We want to offer more flexibility as to formats for data input and output. We also want to use corrections made by the user for improving the computation of profiles, as some form of online learning.

7. REFERENCES

[1] F. Boschetti, M. Romanello, A. Babeu, D. Bamman, and G. Crane. Improving ocr accuracy for classical critical editions. In *Proceedings of the 13th European conference on Research and advanced technology for digital libraries*, ECDL'09, pages 156–167, Berlin, Heidelberg, 2009. Springer-Verlag.

[2] A. Conteh and A. Tzadok. User collaboration in mass digitisation of textual materials. In *Cultural Heritage on line*, pages 83 – 86, 2009.

[3] R. Garfinkel, E. Fernandez, and R. Gopal. Design of an interactive spell checker: optimizing the list of offered words. *Decis. Support Syst.*, 35(3):385–397, June 2003.

[4] R. Holley. How good can it get? analysing and improving ocr accuracy in large scale historic newspaper digitisation programs. *D-Lib Magazine*, (3/4), March/April 2009.

[5] R. Holley. Many hands make light work : Public collaborative ocr text correction in australian historic newspapers. *National Library of Australia Staff Papers*, pages 1–28, 2009.

[6] K. Kukich. Techniques for automatically correcting words in texts. *ACM Computing Surveys*, pages 377–439, 1992.

[7] U. Reffle. Efficiently generating correction suggestions for garbled tokens of historical language. *Nat. Lang. Eng.*, 17(2):265–282, Apr. 2011.

[8] U. Reffle and C. Ringlstetter. Unsupervised profiling of ocred historical documents. *Pattern Recogn.*, 46(5):1346–1357, May 2013.

[9] M. Reynaert. All, and only, the errors: more complete and consistent spelling and ocr-error correction evaluation. In *LREC'08*, pages –1–1, 2008.

[10] M. Reynaert. Non-interactive ocr post-correction for giga-scale digitization projects. In *Proceedings of the 9th international conference on Computational linguistics and intelligent text processing*, CICLing'08, pages 617–630, Berlin, Heidelberg, 2008. Springer-Verlag.

[11] C. Strohmaier, C. Ringlstetter, K. U. Schulz, and S. Mihov. A visual and interactive tool for optimizing lexical postcorrection of OCR results. In *Proceedings of the IEEE Workshop on Document Image Analysis and Recognition, DIAR'03*, 2003.

[12] K. Taghva, J. Borsack, A. Condit, and S. Erva. The effects of noisy data on text retrieval. *Journal of the American Society for Information Science*, 45:50–58, 1994.

[13] K. Taghva, T. Nartker, and J. Borsack. Information access in the presence of ocr errors. In *HDP '04: Proceedings of the 1st ACM workshop on Hardcopy document processing*, pages 1–8, New York, NY, USA, 2004. ACM Press.

[14] K. Taghva and E. Stofsky. OCRSpell: an interactive spelling correction system for OCR errors in text. *International Journal of Document Analysis and Recognition*, 3:125–137, 2001.

[15] R. Wudtke, C. Ringlstetter, and K. U. Schulz. Recognizing garbage in ocr output on historical documents. In *Proceedings of the 2011 Joint Workshop on Multilingual OCR and Analytics for Noisy Unstructured Text Data*, MOCR_AND '11, pages 8:1–8:6, New York, NY, USA, 2011. ACM.

[1] https://github.com/thorstenv/PoCoTo, http://thorstenv.github.io/PoCoTo/